

Professional Summary

Results-driven Python Developer with 4+ years of hands-on experience designing, developing, and deploying scalable microservices, robust REST APIs, and data-driven applications. Strong expertise in Python backend systems, AWS cloud services, SQL, ETL pipelines, and data integration solutions.

- Designed and developed scalable **RESTful APIs and microservices using Python (Flask)** with modular architecture and clean coding standards.
- Built responsive, user-centric web applications using **React, Angular, JavaScript (ES6+), HTML5, and CSS3**.
- Architected microservices-based systems with **API versioning, Swagger/OpenAPI documentation**, and service orchestration best practices.
- Implemented secure authentication and authorization mechanisms using **OAuth2, JWT, and OWASP security standards**.
- Designed and optimized relational and NoSQL databases using **PostgreSQL, MySQL, MongoDB**, with schema migrations managed via **Alembic**.
- Improved system performance through **Redis caching, query indexing, and backend performance tuning techniques**.
- Developed event-driven and asynchronous systems leveraging **Apache Kafka** for real-time data streaming.
- Deployed and managed cloud-native applications on **AWS (EC2, S3, RDS, Lambda, ECS)** ensuring scalability and high availability.
- Implemented **Infrastructure as Code (Terraform)** for automated environment provisioning and consistency.
- Applied containerization and DevOps best practices using **Docker, Kubernetes (fundamentals), and CI/CD pipelines (GitHub Actions, Jenkins, GitLab CI)**.
- Established monitoring and observability solutions using **Prometheus, Grafana, ELK Stack, and AWS CloudWatch**.
- Actively collaborated in **Agile/Scrum environments**, implementing automated testing using **PyTest, Unittest**, and API validation with **Postman**.

TECHNICAL STACK

| Domain       | Technologies                                                                      |
|--------------|-----------------------------------------------------------------------------------|
| Backend      | Python, PySpark, Flask, FastAPI, REST APIs, Microservices, AsyncIO                |
| Frontend     | React, Angular, JS (ES6+), HTML5, CSS3, REST Integration                          |
| DB/Cache     | Snowflake, PostgreSQL, SQL, MySQL, MongoDB, Redis, Alembic                        |
| Cloud/DevOps | AWS (EC2, S3, RDS, Lambda, ECS), Databricks, Docker, Kubernetes, Terraform, CI/CD |
| Messaging    | Kafka, Apache Airflow, Event-Driven Architecture, ETL Pipelines                   |
| Testing      | Pytest, Unittest, Postman, API Testing                                            |
| Tools        | Git, GitHub Actions, Jenkins, Swagger                                             |
| Security/Obs | OAuth2, JWT, AWS CloudWatch, ELK, Prometheus, Grafana                             |

EXPERIENCE:

Molina Healthcare, Inc.  
Software Engineer (Python Developer)

Long Beach, CA  
February 2024 – Present

Responsibilities:

- Architected and developed scalable healthcare applications using **Python (Flask)** in a microservices architecture, integrating with **Java Spring Boot services**, ensuring **HIPAA-compliant secure data handling**.
- Designed and implemented high-performance **RESTful APIs** using Flask, leveraging **PostgreSQL**, optimized indexing, and **Alembic-based schema versioning** for transactional claims and eligibility systems.
- Built modular backend components following **SOLID principles, OOP, and clean architecture patterns**, supporting 5M+ Medicaid users in a high-availability production environment.
- Implemented **Infrastructure as Code (IaC)** using **Terraform**, provisioning AWS resources (EC2, Lambda, IAM, VPC) for scalable, fault-tolerant deployments.
- Developed event-driven data pipelines using **Python automation scripts**, integrating **AWS S3, Lambda, EventBridge**, and asynchronous workflows for real-time ingestion and transformation.
- Integrated centralized logging and observability using **ELK Stack (Elasticsearch, Logstash, Kibana)** and **AWS CloudWatch**, improving MTTR by 40% in production systems.
- Engineered secure authentication mechanisms using **OAuth2, JWT tokens, RBAC**, and implemented API-level security aligned with OWASP standards.
- Led backend development for a conversational AI chatbot service, designing scalable API endpoints, managing session state, and optimizing request-response latency in distributed environments.
- Conducted backend architecture reviews, identifying scalability bottlenecks, improving API throughput, and recommending **horizontal scaling strategies** in AWS.
- Mentored engineers on **Python best practices, REST API standards, microservices communication patterns, CI/CD workflows, and secure cloud deployments**.

Skillset: **Python (Flask, FastAPI), REST APIs, Microservices, OOP, AsyncIO, PostgreSQL, MySQL, MongoDB, Redis, Alembic, Kafka, AWS (EC2, S3, RDS, Lambda, ECS), Docker, Kubernetes, Terraform, CI/CD, React, JavaScript (ES6+), OAuth2, JWT, ELK, Prometheus, Git.**

Responsibilities:

- Engineered RESTful backend APIs using **Python and Flask**, enabling real-time industrial telemetry tracking and rule-based alert orchestration.
- Implemented modular service architecture with **Blueprints, WSGI, and middleware layers** to support scalable manufacturing dashboards.
- Managed database schema migrations using **Alembic and SQLAlchemy ORM**, ensuring version-controlled deployments and backward compatibility.
- Designed optimized relational data models (PostgreSQL/MySQL) with indexing, constraints, and query performance tuning.
- Integrated AWS services (**S3, EC2, CloudWatch, IAM**) to build secure, event-driven data ingestion and monitoring pipelines.
- Developed asynchronous background jobs using **Celery and Redis**, automating scheduled data refresh and alert processing tasks.
- Created cron-based automation scripts and self-healing monitoring utilities using **Python multiprocessing and logging frameworks**.
- Refactored monolithic Java components into lightweight **Flask-based microservices**, improving API latency and maintainability.
- Built event-driven notification engine leveraging **Python decorators, custom rule evaluators, and JSON-based configuration parsing**.
- Implemented unit and integration testing using **PyTest, unittest, and mock libraries**, increasing backend test coverage.
- Optimized API performance with caching strategies using **Redis and Flask-Caching**, reducing response time for high-volume endpoints.
- Collaborated in Agile workflows (Sprint Planning, Code Reviews, CI/CD pipelines) integrating **Git, Docker, and AWS deployment strategies**.

**Skillset:** Python, Flask, SQLAlchemy, Alembic, REST APIs, Celery, Redis, AWS (S3, EC2, CloudWatch, IAM), PostgreSQL, Docker, Git, PyTest, CI/CD, Microservices Architecture, Cron Automation

Responsibilities:

- Developed foundational backend components for engineering test tools using **Python, Flask, and MySQL**, following Agile and SDLC best practices.
- Built and enhanced secure **RESTful APIs in Python (Flask)** with token-based authentication and role-based access control for Angular-based dashboards.
- Implemented backend business logic using **Python OOP principles, modular architecture, and reusable service layers**.
- Designed and executed unit and integration tests using **Python unittest and PyTest frameworks**, improving pre-release defect detection.
- Wrote optimized **SQL queries and stored procedures** for high-volume backend data extraction, validation, and reporting accuracy.
- Automated backend validation scripts using **Python scripting and logging modules** to streamline engineering test data processing.
- Created API documentation using **Swagger/OpenAPI specifications**, improving developer onboarding and cross-team integration.
- Participated in backend prototyping using Python-based microservices to validate system scalability and technical feasibility.
- Developed system health monitoring and reporting automation using **Python cron jobs and scheduled scripts**.
- Assisted in debugging and performance tuning of Flask applications using profiling and logging tools.
- Collaborated with frontend teams to structure clean JSON responses and ensure seamless API consumption.
- Conducted technical walkthroughs and knowledge transfer sessions, explaining Python backend workflows and API integrations to QA and support teams.

**Skillset:** Python, Flask, REST APIs, MySQL, SQL Optimization, PyTest, unittest, Swagger/OpenAPI, OOP, Microservices, Cron Jobs, JSON, Agile, Git

PROJECTS:

**E-Commerce Order Management Platform | Java, Spring Boot, Go, Python, Flask, PostgreSQL, Redis, Docker, AWS (ECS, S3, RDS), Apache Kafka, Terraform, Prometheus, Grafana**

- Developed backend REST APIs and microservices using Python (Flask and FastAPI), deploying containerized services to AWS ECS with Terraform following Infrastructure as Code best practices.
- Architected ETL pipelines and asynchronous workflows using Python, PySpark, and Apache Kafka to enable real-time processing of large datasets and secure transactional updates.
- Designed and optimized database schemas in PostgreSQL and Snowflake using Alembic, leveraging complex SQL query optimization to reduce query latencies by 45% during peak loads.
- Orchestrated end-to-end data pipelines using Apache Airflow and instrumented application performance observability with AWS CloudWatch to ensure high availability of core APIs.

**Community Connect – Blockchain App | Solidity, React, Web3.js, Ethereum, IPFS, Truffle**

- Engineered data integration services and secure backend systems using Python to process cryptographically signed payloads and coordinate distributed application states.
- Built responsive frontend applications using React, integrating secure RESTful APIs to handle user authorization, data inputs, and transactional interactions.
- Developed Python data processing scripts to manage high-integrity database writes, ensuring reliable and tamper-proof storage of application event logs.
- Conducted end-to-end performance optimization and query tuning across PostgreSQL databases to minimize resource consumption and maximize data retrieval speeds.

EDUCATION:

